

Laurentian University
Université Laurentienne

**SCHOOL OF ENGINEERING AND COMPUTER SCIENCE**

# Cache Performance Case Study: How Cache Design Shapes Processor Performance in AMD's Ryzen 9000 Series

**COSC 3406 – Computer Organization**

**Faculty Advisor:**
Prof. Williams Nwadiugwu

Fall, 2026.

**Submitted By Team 2**


| Name | Student ID |
|---|---|
| Abu Ishaque Md Israil | 0449749 |
| Chowdhury Shafquat Ahmed Nibraj | 0442248 |


**October 21, 2026**

# Chapter 1 – Project Synopsis, and Literature Review

## 1.1 Project Synopsis

The current case study focuses on the issue of cache memory and tries to evaluate the impact of the design of this type of memory on the performance of a processor. Firstly, one needs to point out the discrepancy between the speed of CPU operation and main memory slowness. Then it will be helpful to consider the methods applied by the cache to address the issue including such concepts as locality, mapping, replacement and write policies. In terms of real-world application, we decided to use the example of AMD's Ryzen 7 9700X (Zen 5) comparing it with both Ryzen 7 7700X and Ryzen 7 9800X3D. The research used to conduct the case study includes two scientific papers. The first one is by Jaleel et al. (2010) and it is devoted to the decision process in the question of removal of particular lines from the cache. The second paper by Gober et al. (2022) focuses on ChampSim – cache simulator utilized by researchers for testing purposes. According to our worked examples and the AMAT formula, the proper caching hierarchy is able to reduce the average memory access from approximately 300 cycles to 11. It is important to note that the majority of decisions in terms of the cache design have a trade-off in terms of speed, price and energy usage. It is necessary to admit that we did not conduct any tests using hardware. The data presented is either taken from AMD documentation and the two papers mentioned above or is just made up as an example.

Our research goals were several. At first, we had to discuss the reasons for existence of caches and their functioning and then consider the design of the cache of a real-world processor comparing it with other processors. Then, we needed to analyze the cache performance in terms of the hit rate, miss rate, AMAT and CPI. Finally, we needed to discuss the trade-offs, in particular, related to the replacement policy using the two research papers as evidence.

## 1.2 The Memory Wall: Why Processors Need Cache

The development of processors was much faster than that of memory; our textbook calls this phenomenon the performance balance problem. Modern processors cores are able to operate at frequencies above 5 GHz and execute several instructions per cycle, whereas it requires tens of nanoseconds to fetch one piece of information from DRAM. For instance, it takes about four clock cycles for L1 cache of 9700X to fetch a piece of information; it takes 14 clock cycles to fetch information from L2 cache and around 50 clock cycles from L3 cache, but it takes around 300 clock cycles to fetch information from main memory.

We will demonstrate an example to understand the importance of the problem described above. Let us consider the program containing 100 instructions which takes 100 clock cycles if memory accesses are instantaneous. One instruction in this program takes 300 cycles to be

executed because of its access to main memory; hence, the whole program takes 400 clock cycles and the processor wastes 75% of time because of one slow access.

You might suggest using fast memory to construct the main memory of a computer, but it is too expensive. Memory used for construction of caches consists of SRAM, which requires six transistors for storing one bit, whereas DRAM uses one transistor and one capacitor to store one bit of information. Thus, the concept of cache lies in the usage of small amount of fast memory and its emulation as the big amount of fast memory. It depends on design options which we are going to discuss in our further report.

## 1.3 Literature Review: Cache Replacement with RRIP

In the paper by Jaleel et al. (2010), the authors discuss the replacement policy which is a cache algorithm for determining which lines should be replaced in case there is no space left in the sets. Most caches use the LRU (Least Recently Used) replacement strategy or its modification. In the article, LRU works well as long as the program reuses the recently referenced data, but in large last-level caches it encounters difficulties in two cases. The first one is called a scan and it refers to the process when the program reads a large amount of data at once and this data pushes out the necessary data from the cache. The second problem is called thrashing and it is caused by reading data in loops slightly larger than the cache. In such a situation, the program continues replacing the exact line which is going to be used.

The proposed solution of the authors is called Re-Reference Interval Prediction (RRIP). Unlike LRU that tracks the time since the line was accessed, RRIP assigns a small counter that predicts when it is going to be used again. Such a counter consumes only 2 bits per line. As soon as the new line appears in the cache, it receives a 'long' prediction until it accesses it again. The authors describe the static version of RRIP (SRRIP), which is effective against scans, the bimodal version of RRIP (BRRIP), which is effective against thrashing, and dynamic RRIP (DRRIP) which changes its behavior automatically depending on the situation and uses set dueling approach to do that. The paper helped us understand that replacement policy may consume less hardware resources compared to LRU and cope with the above-mentioned problems effectively.

.

## 1.4 Literature Review: The ChampSim Simulator

The second paper, by Gober et al. (2022), is about ChampSim, which is a free and open-source simulator from Texas A&M University. Since building a real chip takes years, researchers use simulators like this to test ideas such as RRIP first. ChampSim is trace-driven, meaning a real program is first run with a tracing tool that records all of its instructions and

memory addresses, and then the simulator plays that recording back through a model of the processor. In ChampSim, the replacement policy of each cache can be swapped out like a plug-in, and the output is given as IPC (instructions per cycle). The authors also explain what the simulator can't do. At the time the paper was written, for example, it did not model cache coherence and it only supported non-inclusive caches.

### 1.5 How the Two Papers Fit Together

However, when the two studies are combined, both aspects of the question are addressed. The first paper, by Jaleel et al. (2010), provides the example of a more efficient design of cache memory, while Gober et al. (2022) demonstrate a way of testing such a design objectively. Neither paper, however, mentions AMD's Ryzen 9000 series processors or addresses cost-related issues.

## Chapter 2 – System System/architecture diagram

### 2.1 The Case-Study Processor and Its Cache Hierarchy

The processor we chose for this case study is the AMD Ryzen 7 9700X. It is an 8-core, 16-thread desktop processor from the Ryzen 9000 series, released in August 2024 and built on AMD's Zen 5 core. Figure 1 shows its memory hierarchy as seen from one core. Each core has its own 32 KB L1 instruction cache, 48 KB L1 data cache and 1 MB L2 cache, and according to AMD's documentation all eight cores share a single 32 MB L3 cache. The cache line size is 64 bytes at every level.

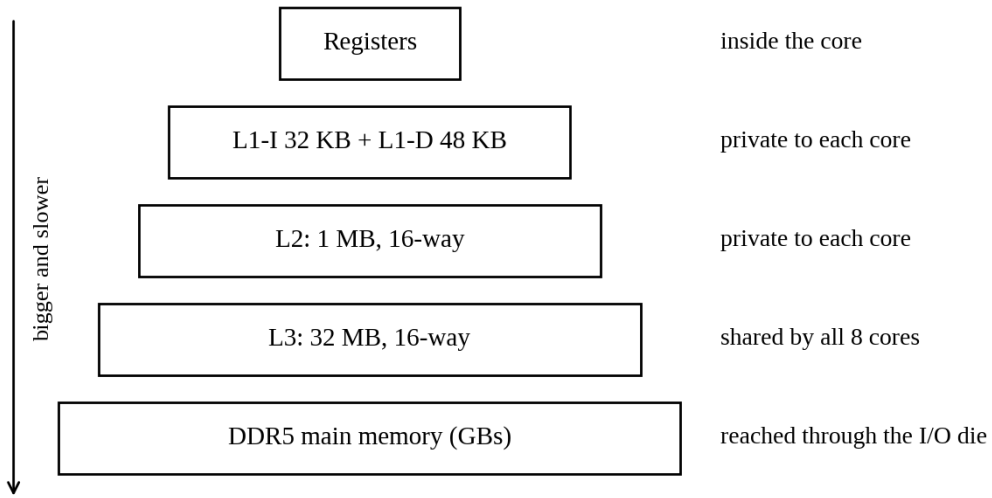



*Figure 1. The cache hierarchy of the Ryzen 7 9700X, seen from one core (our sketch).*

The L1 cache is divided into an instruction cache and a data cache, allowing fetching of an instruction along with loading of data in a single cycle without interference between them, while L2 and L3 caches have both kinds of lines stored inside. What caught us by surprise was that AMD uses its L3 cache primarily as a victim cache. It means that lines are added to L3 only when evicted from L2, and not during their initial load, thus L3 is essentially a big storage of data lines that the core will most likely need once again. Meanwhile, the cores and the L3 cache are situated on the chiplet, while the DDR5 memory controller is located on the I/O die. That makes a miss in L3 cache go through Infinity Fabric to the I/O die in order to eventually reach the main memory.

## 2.2 Inside Each Cache and How an Address Is Split

All the caches have the similar set of components. They all have a data array that stores the 64-byte lines and a tag array, which tracks the address of each line along with valid and dirty bits. A cache is also comprised of comparators that compare all tags in one set simultaneously, some bits in each set used for the replacement policy and a controller that handles all misses. As soon as the cache gets the address, it splits it into three parts: the tag, index and offset. The values of these fields were calculated for the Zen 5 L1 data cache with the size of 48 KB and 12 way associativity, consisting of 64-byte lines and 48-bit addresses:

Lines = 48 KB ÷ 64 B = 768. Sets = 768 ÷ 12 ways = 64. Offset = $\log_2$(64 B) = 6 bits. Index = $\log_2$(64 sets) = 6 bits. Tag = 48 − 6 − 6 = 36 bits.

While doing this calculation we noticed something interesting. The index and the offset add up to 12 bits, which is the same size as one 4 KB memory page. Because of this, the cache can start choosing the set while the TLB is still translating the address, so the two lookups can happen at the same time. Zen 4's L1 worked the same way, since 32 KB divided into 8 ways also gives 4 KB per way. So when AMD increased the L1 size by 50% in Zen 5, they added more ways (from 8 to 12) and not more sets. This keeps each way at 4 KB, and we think this is likely one of the main reasons the latency stayed at 4 cycles.

## 2.3 Full Technologies/Tools Stack and System Requirements

Table 1 lists all the technologies and tools that were part of this case study, from the processor we studied to the tools we used for our analysis.

*Table 1. Full technologies and tools stack for this case study.*

| Layer | Technology / tool | Role in our study |
|---|---|---|
| Processor | AMD Ryzen 7 9700X (Zen 5), 8 cores | Main real-world case-study system |
| Cache hierarchy | L1-I 32 KB, L1-D 48 KB, L2 1 MB per core; L3 32 MB shared | The architecture we analyze |
| Interconnect and memory | Infinity Fabric, I/O die, dual-channel DDR5 | Path taken by every L3 miss |
| Replacement policy | LRU, SRRIP, BRRIP, DRRIP (Jaleel et al., 2010) | Compared in Chapter 3 |
| Simulator | ChampSim in C++, with its tracer and a JSON configuration file (Gober et al., 2022) | How cache designs are tested in research |
| Performance metrics | Hit rate, miss rate, AMAT, CPI, IPC | Our quantitative analysis |
| Our own analysis | Python with Matplotlib; Microsoft Word | Calculations, figures and this report |

According to Gober et al. (2022), for anyone wishing to conduct an experiment similar to ours using ChampSim, the requirements include having an updated version of the C++ compiler and Python. At the time the paper was written, this implied the use of GCC 7, Clang 4, or Visual Studio 2017 alongside Python 3.6, which could easily be done by any standard student laptop. There must be a trace file that could either be obtained from ChampSim's own tracer or elsewhere online. Finally, the 9700X requires AM5 board and DDR5 RAM.

## 2.4 Technical Comparison Table

In Table 2 we compare our processor with the chip that came before it and with its large-cache version. We come back to two of these differences in Chapter 3, which are the extra L1 ways in Zen 5 and the much larger L3 in the 9800X3D.

*Table 2. Cache comparison of three 8-core AMD desktop processors (from AMD's published documentation).*

| Feature | Ryzen 7 7700X (Zen 4) | Ryzen 7 9700X (Zen 5) | Ryzen 7 9800X3D (Zen 5) |
|---|---|---|---|
| L1 instruction (per core) | 32 KB | 32 KB | 32 KB |
| L1 data (per core) | 32 KB, 8-way | 48 KB, 12-way | 48 KB, 12-way |
| L1 data latency | 4 cycles | 4 cycles | 4 cycles |
| L2 (per core) | 1 MB, 8-way | 1 MB, 16-way | 1 MB, 16-way |
| L3 (shared) | 32 MB | 32 MB | 96 MB (32 MB + 64 MB stacked) |
| Cache line size | 64 bytes | 64 bytes | 64 bytes |

# Chapter 3 – Critical evaluation and Real-world case study

## 3.1 Case-Study System Description

### *Locality*

The reason caches are valuable is that memory accesses are not random by the program. Temporal locality occurs if the program accesses an entity and then accesses it again later, such as in a loop counter. Spatial locality occurs if the program accesses an entity and then accesses other entities around the same address, for instance accessing elements in an array. Here is a simple calculation that illustrates the benefit of spatial locality. A single 64 bytes block will contain 16 four bytes integers, and hence when looping over an array and summing its elements, only the first integer in each block is a miss while the rest of the 15 are already in the cache.

### *The Lookup Path, Hits and Misses*

Figure 2 shows the steps the core goes through each time it reads from memory. It looks in L1 first, and if the data is there, which is called a hit, it gets the data back in about 4 cycles. If the data is not there, which is a miss, it checks L2, then L3, and then main memory. When the data finally comes back, the whole 64-byte line is copied into the cache so that the next access to a nearby address will be a hit.

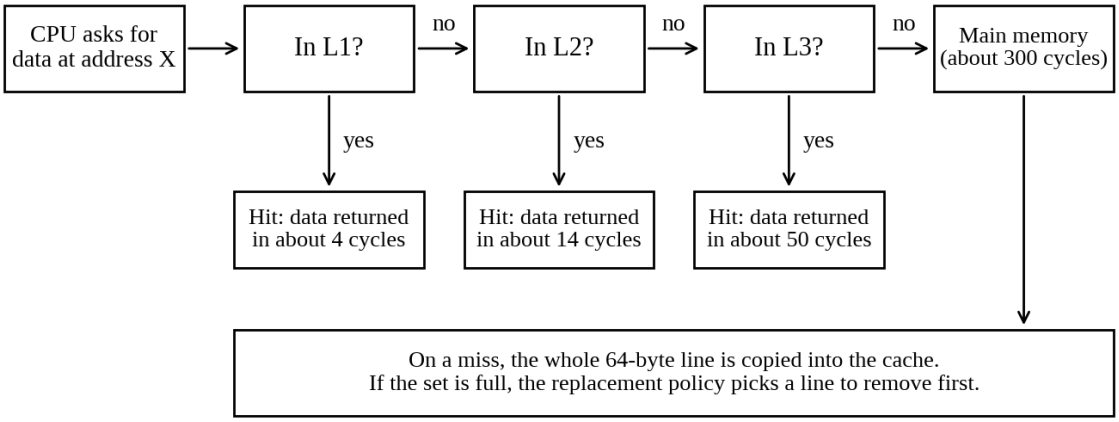



*Figure 2. What happens on a memory read (our sketch).*

$$\textit{Hit rate} = \textit{hits} \div \textit{total accesses} \qquad \textit{Miss rate} = 1 - \textit{hit rate}$$

For example, if a program makes 1,000 memory accesses and finds the data in the cache 900 times, the hit rate is 90% and the miss rate is 10%. This is only an example and not a measurement, but we use it again in Section 3.2.

### *Mapping: Where a Block Is Allowed to Go*

However, a cache requires a policy that dictates where a block of memory is to be stored. It seemed more intuitive for us to view the caching issue as one relating to parking lots. In direct mapping, a car has one spot only for parking, making the retrieval fast, but if two cars are assigned the same spot, they constantly kick each other out of there. In fully associative mapping, a car may park anywhere in the lot, but locating it requires checking all spots in the parking lot. With the set-associative mapping, however, the plate number dictates a row in the lot for the car and it may park at any spot in the row. All AMD's caches apply this technique.

For demonstration purposes, the same sequence of eight memory accesses is used in two very small caches, capable of storing 4 lines each, as presented in Table 3 below. For the direct-mapped cache, block b stores in line (b mod 4). The 2-way cache consists of 2 sets, block b is assigned set (b mod 2) and LRU is applied within each set.

*Table 3. The same access pattern in two caches of the same size (calculated by us).*

| Step | Block | Direct: line | Direct: result | 2-way: set | 2-way: result |
|---|---|---|---|---|---|
| 1 | 0 | 0 | Miss | 0 | Miss |
| 2 | 4 | 0 | Miss (evicts 0) | 0 | Miss |
| 3 | 0 | 0 | Miss (evicts 4) | 0 | Hit |
| 4 | 4 | 0 | Miss (evicts 0) | 0 | Hit |
| 5 | 1 | 1 | Miss | 1 | Miss |
| 6 | 0 | 0 | Miss (evicts 4) | 0 | Hit |
| 7 | 2 | 2 | Miss | 0 | Miss (LRU evicts 4) |
| 8 | 0 | 0 | Hit | 0 | Hit |
| Total | | | 1 of 8 (12.5%) | | 4 of 8 (50%) |

Even though both caches have the same amount of storage, the 2-way cache got four times more hits. In the direct-mapped cache, blocks 0 and 4 both go into line 0, so they keep removing each other, and these are known as conflict misses. In the 2-way cache, both blocks can stay in the same set together.

### *Replacement: LRU and RRIP*

When a set is full, the replacement policy has to choose which line to remove. LRU removes the line that has gone the longest without being used. Jaleel et al. (2010) explain that this does not work well with scans and thrashing, because LRU always treats the newest line as the most important one. RRIP avoids this by giving each line a 2-bit value called the RRPV, which goes from 0, meaning the line will probably be used again soon, to 3, meaning it probably won't be used for a long time. Figure 3 shows how this works.

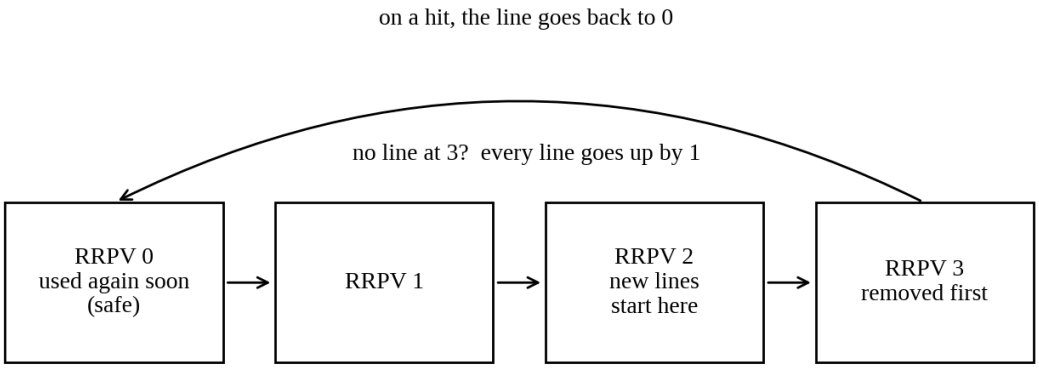



*Figure 3. How RRIP manages one cache set, based on Jaleel et al. (2010) (our sketch).*

The rules for RRIP are quite simple. The new line begins from 2, and if it scores a hit then it is reset to 0. If the time comes to remove the line, then the cache selects the line which is in state 3, and in case there is no such line, it increases all the lines in the set by 1 and looks again. As the data scanned was used just once and thus it never becomes 0 and quickly reaches 3 and gets removed, and the other lines which are used repeatedly remain in the 0th state. DRRIP implements one step more – set dueling where certain sets use SRRIP, some use BRRIP, and the remaining cache uses the policy with less misses.

### *Write Policy*

There are two write policies that exist. The write-through means that all writes go to the cache and the memory, and write-back means that the write goes just to the cache and the block is flagged as dirty, and main memory is written to only when the block is removed from the cache. For instance, in case when a loop updates the same counter for 1000 times, then write-through will result in 1000 writes, and the write-back in just one. This is the reason why write-back is used in modern caches, including AMD.

## 3.2 Quantitative Analysis and Performance Metrics

**System Results/Output Constraints/Parameters:**

In our calculations we used AMD's figures for the L1 and L2 hit times of the 9700X, which are 4 and 14 cycles, and about 50 cycles for L3, which is around the usual value for this type of chip. The other values are examples we chose, because real miss rates can be very different from one program to another. We used 300 cycles for main memory, local miss rates of 10% for L1, 40% for L2 and 30% for L3, and a clock speed of about 5 GHz to convert cycles into nanoseconds.

### *i) Operational/Transitional Efficiency*

The main measurement we used is the average memory access time (AMAT):

$$AMAT = Hit\ time + Miss\ rate \times Miss\ penalty$$

**One cache level:** Using the example from earlier, with a 4-cycle hit, a 10% miss rate and a 300-cycle miss penalty, AMAT = 4 + 0.10 × 300 = 34 cycles. If the hit rate goes up to 99%, AMAT = 4 + 0.01 × 300 = 7 cycles. A change from 90% to 99% does not sound like much, but it makes memory almost five times faster on average, which surprised us.

**The full 9700X hierarchy:** With these values we worked from the bottom level up, because the miss penalty of each level is the AMAT of the level below it:

L3: 50 + 0.30 × 300 = 140 cycles. L2: 14 + 0.40 × 140 = 70 cycles. L1: 4 + 0.10 × 70 = 11 cycles.

This means a memory access takes about 11 cycles on average, which is around 2.2 ns at 5 GHz. Without any cache it would take about 300 cycles, or 60 ns, so the cache hierarchy makes memory access about 27 times faster.

**Effect on CPI:** In the textbook formula $T = Ic \times CPI \times \tau$, cache misses add extra cycles to the CPI. Let's say a program has a base CPI of 1, 30% of its instructions are loads or stores, 2% of those miss in every cache level, and each miss costs 300 cycles. The extra cycles per instruction would be 0.30 × 0.02 × 300 = 1.8, which makes the CPI 2.8. If a better cache lowered that miss rate to 1%, the extra cycles would drop to 0.9 and the CPI to 1.9, so the same program would run about 32% faster without changing the clock speed. In ChampSim's terms, the IPC would go from about 0.36 to about 0.53.

**Replacement-policy efficiency:** Storage requirements for each policy were also considered. In case of 16-way set, LRU can be designed using an age of 4 bits per line, amounting to 64 bits per set. RRIP, on the other hand, requires only 2 bits per line; therefore, the amount of storage used per set is 32 bits. This shows that RRIP requires only half the storage compared to LRU.

### *ii) Performance, Scalability, Cost, and Energy Implications*

**Performance:** From Zen 4 to Zen 5, AMD increased the L1 data cache by 50% and raised its associativity from 8 to 12 ways while keeping the latency at 4 cycles. They also doubled the associativity of the L2 from 8 to 16 ways, which mostly helps reduce conflict misses. These changes do not make the clock any faster, but more accesses end up hitting in the cache, and that is where the extra speed comes from.

**The 9800X3D trade-off:** The 9800X3D adds a stacked 64 MB cache die, giving it 96 MB of L3 in total. Let's assume this lowers the L3 miss rate from 30% to 15% in a game, but adds about 4 cycles to each L3 hit (this is our own assumption). Then L3 = 54 + 0.15 × 300 = 99, L2 = 14 + 0.40 × 99 ≈ 53.6, and L1 = 4 + 0.10 × 53.6 ≈ 9.4 cycles, which is about 15% better than 11. However, for a program whose data already fits in 32 MB, the miss rate stays at 30%, and the same calculation gives about 11.2 cycles, which is slightly worse. So whether

the bigger cache helps or not depends on the program. On this chip AMD also placed the extra cache underneath the cores so that the cores are closer to the cooler, which shows that even the physical position of the cache is a trade-off.

**Scalabilit:** Since all eight cores in the 9700X share one L3, a program that uses a lot of memory on one core can push out data that the other cores still need. When cores share data, their copies have to be kept consistent using a cache coherence protocol such as MESI, and the more cores there are, the more coherence traffic there is. On the 16-core 9950X, each of the two chiplets has its own L3, and data shared between the two has to go through the I/O die, which is slower. On top of that, two DDR5 channels have to supply data to all the cores. Jaleel et al. (2010) also point out that shared last-level caches are where LRU struggles the most, because a scan running on one core can remove data that other cores still need.

**Cost:** Most of the cost of a cache comes from the chip area it takes up. As a rough calculation, the 32 MB L3 is about 268 million bits, and at 6 transistors per bit that is around 1.6 billion transistors just for storing the data, without counting tags or control logic. This is why designers cannot keep making caches bigger, and why the extra 64 MB in the 9800X3D is on a separate stacked die, which makes it more expensive to package. RRIP is a good example of an improvement that does not cost much, because it gets better results out of the same cache while using less storage than LRU.

**Energy:** Reading from main memory takes a lot more energy than reading from a small cache on the chip, because the request has to leave the chip, go to the DRAM and come back. So every cache hit saves both time and power. On the other hand, bigger caches with more ways use more energy on each access, because more tags need to be checked and the signals have to travel further.

**Trade-offs in short:** Every design choice we looked at has a good side and a cost. A bigger cache gives fewer misses but slower hits and a higher cost, and more ways give fewer conflict misses but use more energy on each access. A shared L3 lets cores use space that the others are not using, but the cores also have to compete for it. Even RRIP is only a prediction, which is why DRRIP keeps testing two policies and switches between them.

# Chapter 4 – Conclusion

## 4.1 Conclusion

The initial problem statement - the effect of cache design on processor performance - received its clear explanation in terms of AMAT. As the result of the implementation of three

cache levels in the 9700X, the average memory access decreases from 300 cycles to 11 cycles, which means 27 times faster access. The small-scale examples used above helped understand how such increase occurs: spatial locality increases hit rate to 94%, 2 way cache gives four times more hits compared to direct-mapped cache of equal capacity, while write-back reduces memory write accesses by 99.9% in the presented example.

The two reviewed papers provided us with some valuable insights, which could not be discovered otherwise. First, Jaleel et al. (2010) prove that replacement policy is important and RRIP is superior to LRU for handling scans and thrashing with half of the storage requirements. Second, Gober et al. (2022) showed how similar designs are tested in simulators such as ChampSim where only one variable is changed at once in order to provide valid results of comparison. The comparison of 7700X, 9700X and 9800X3D revealed that every cache design choice has its price. Larger L1 requires more ways to maintain its high performance, and enormous L3 caches are only helpful if the data cannot be fitted into smaller L3.

There are certain limitations associated with this project. All the data used in this project were taken either from AMD directly, from the papers mentioned above or from our own example and labelled, and no simulation or testing of real hardware was conducted. The real-life miss rates depend a great deal on the particular program. If we had more time, our further steps in this project would include the implementation of the 9700X cache in ChampSim and comparing the performance of LRU, SRRIP and DRRIP using traces of actual programs.

This project has completely transformed our understanding of processor performance. Instead of relying solely on clock speed and the number of cores when comparing processors, now we know that an important portion of performance is achieved by supplying the core with necessary data, and it is largely determined by cache design.

## 4.2 References

Gober, N., Chacon, G., Wang, L., Gratz, P. V., Jiménez, D. A., Teran, E., Pugsley, S., & Kim, J. (2022). *The Championship Simulator: Architectural simulation for education and competition* (arXiv:2210.14324). arXiv. https://doi.org/10.48550/arXiv.2210.14324 (Source code: https://github.com/ChampSim/ChampSim)

Jaleel, A., Theobald, K. B., Steely, S. C., & Emer, J. (2010). High performance cache replacement using re-reference interval prediction (RRIP). In *Proceedings of the 37th Annual International Symposium on Computer Architecture (ISCA '10)* (pp. 60–71). ACM. https://doi.org/10.1145/1816038.1815971